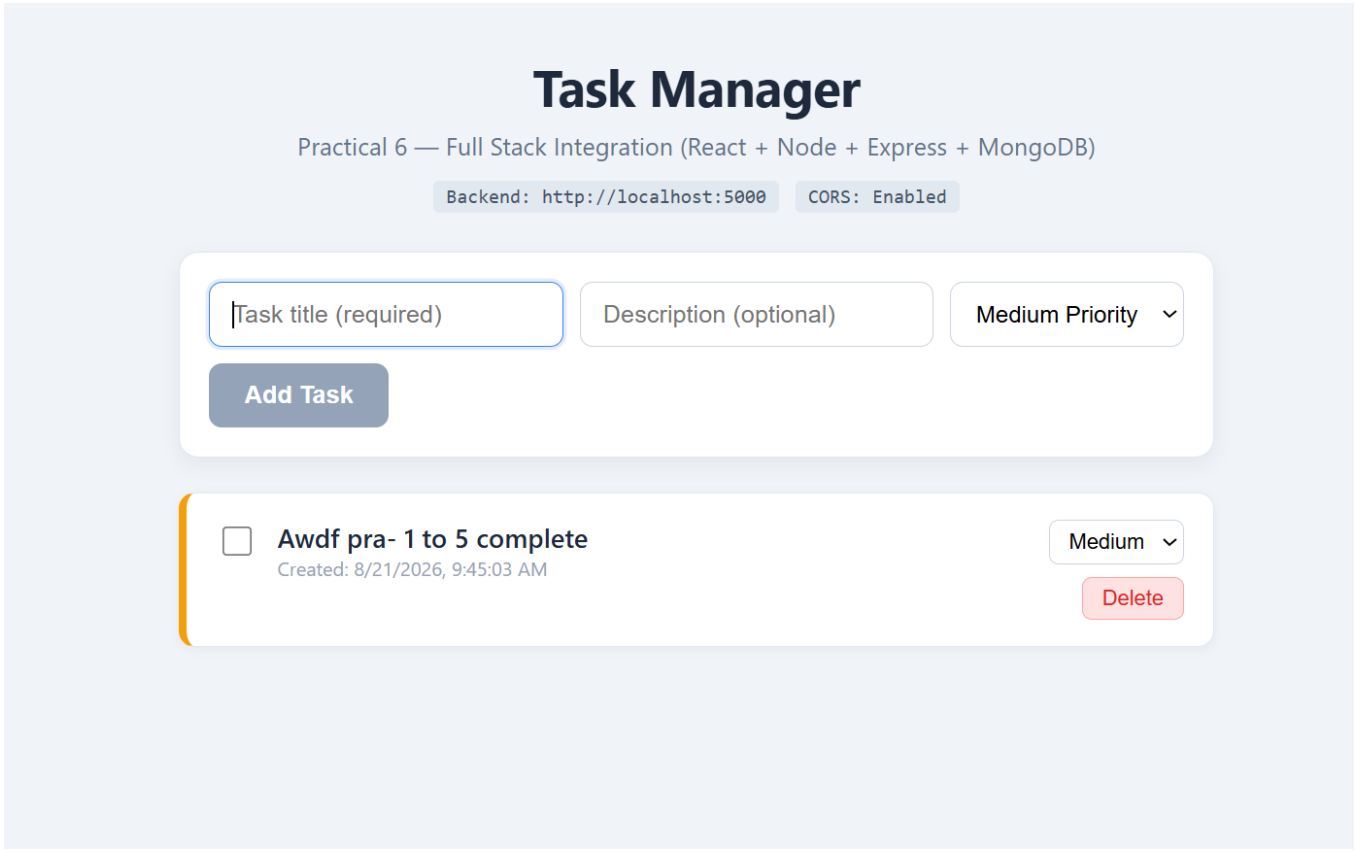
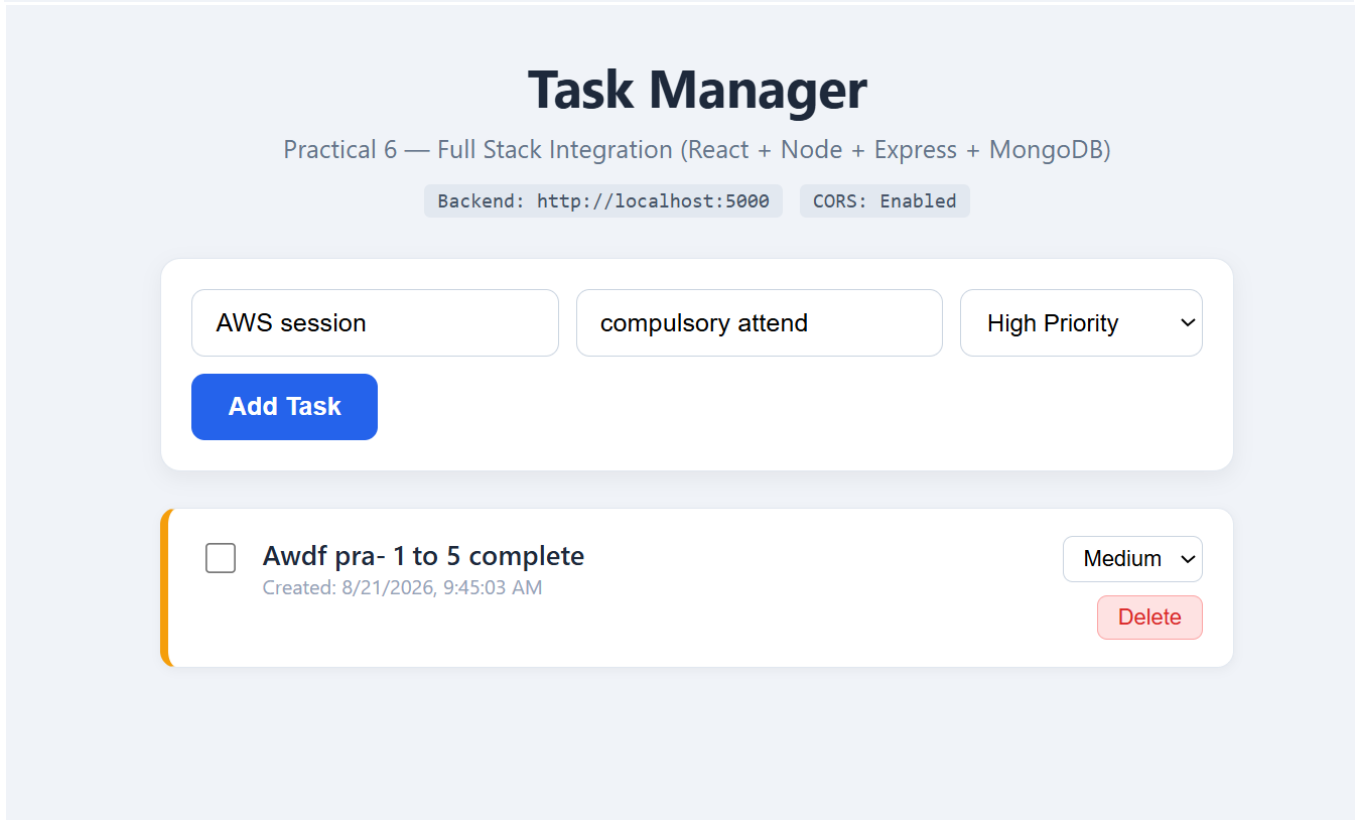
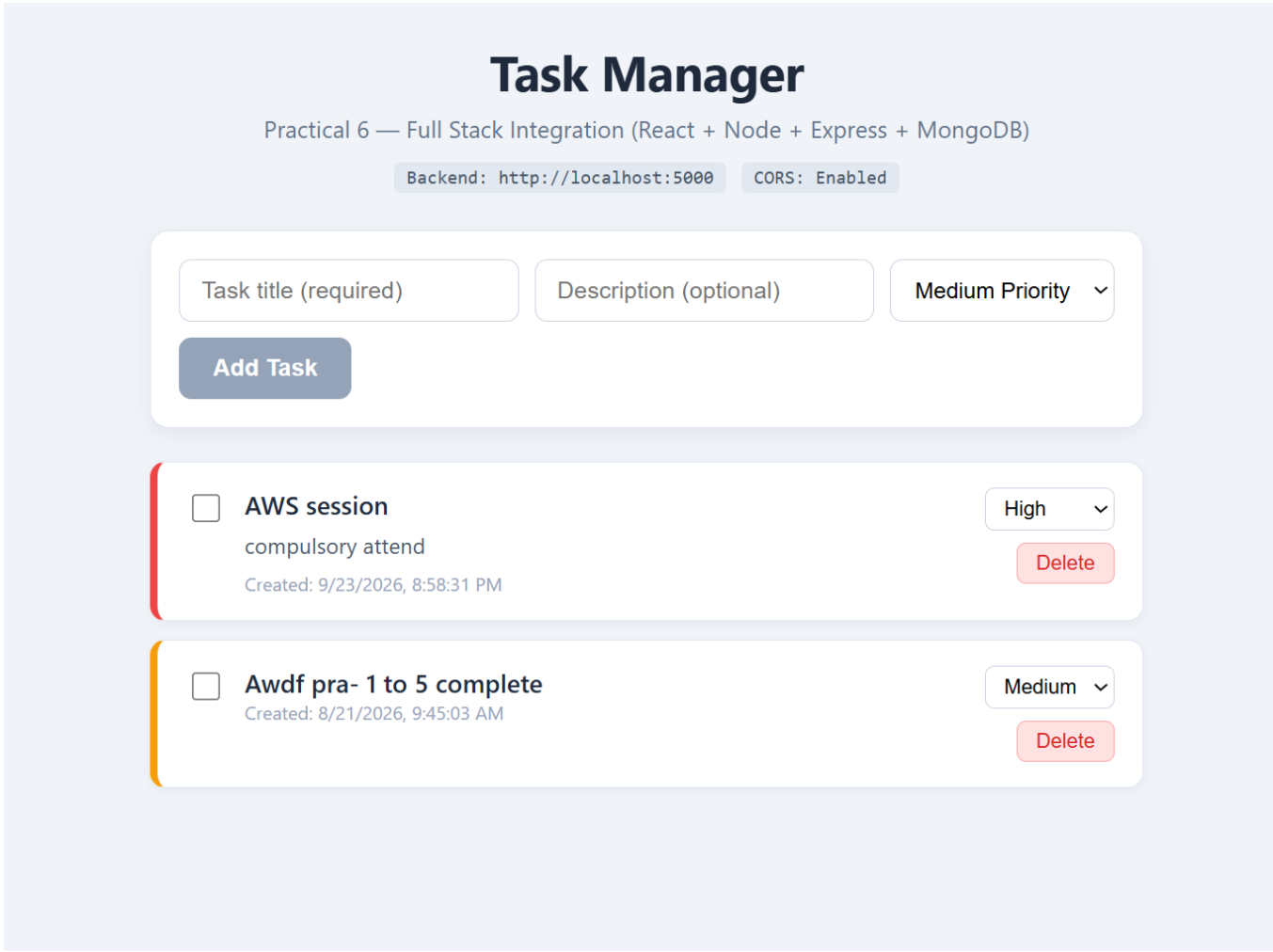
Practical 6: Full Stack Integration React + Node + MongoDB

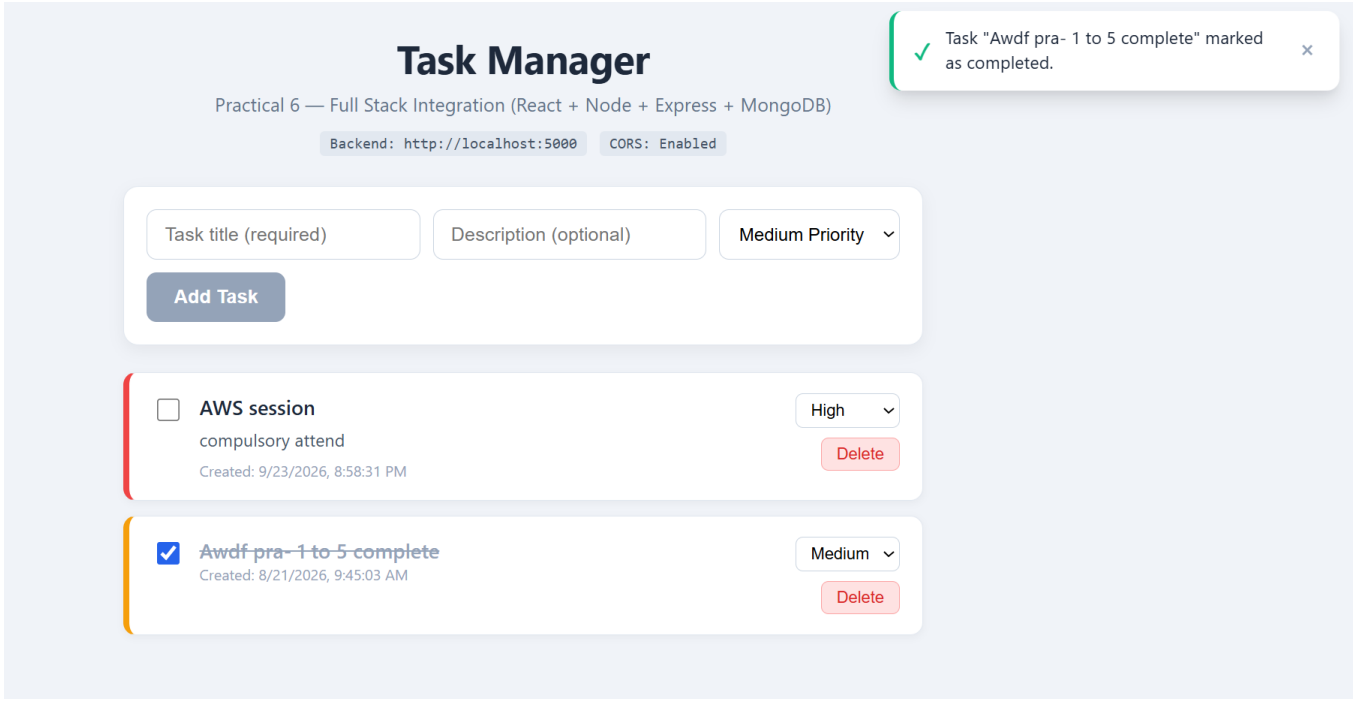
Objective	To wire the React frontend to the Node/Express/MongoDB backend into a fully functional full-stack application.
Problem Statement	• Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5. • Replace the GitHub repo data from Practical 3 with task data from your own backend. • The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls. • All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain. • Handle loading and error states for every API interaction, not just the initial fetch.
Diagram	React Frontend (localhost:5173) | fetch/axios calls v Express Backend (localhost:5000) | Mongoose v MongoDB Database Flow: UI Form → POST /tasks → MongoDB → UI re-fetches → updated list rendered

	1. Install and configure CORS on the Express backend:
	npm install cors
	const cors = require('cors');
	app.use(cors());
	2. In React, create a central api.js file with the base backend URL:
	const BASE_URL = 'http://localhost:5000';
	export const getTasks = () => fetch(`${BASE_URL}/tasks`).then(res
	=> res.json());
	3. Replace the Practical 3 GitHub fetch logic with calls to your own /tasks endpoint.
	4. Build a task creation form that POSTs to /tasks and updates local state on success.
	5. Implement update (PUT) and delete (DELETE) actions, each followed by a state update or re-fetch.
	6. Wrap each API call in proper loading/error handling, reusing the pattern from Practical 3.
	7. Test the complete flow: create → view → update → delete, refreshing the browser to confirm persistence.


Task Manager
Practical 6 — Full Stack Integration (React + Node + Express + MongoDB)
Backend: http://localhost:5000
CORS: Enabled
Task title (required)
Description (optional)
Medium Priority
Add Task
Awdf pra- 1 to 5 complete
Created: 8/21/2026, 9:45:03 AM
Medium
Delete


Task Manager
Practical 6 — Full Stack Integration (React + Node + Express + MongoDB)
Backend: http://localhost:5000
CORS: Enabled
AWS session
compulsory attend
High Priority
Add Task
Awdf pra- 1 to 5 complete
Created: 8/21/2026, 9:45:03 AM
Medium
Delete


Task Manager
Practical 6 — Full Stack Integration (React + Node + Express + MongoDB)
Backend: http://localhost:5000
CORS: Enabled
Task title (required)
Description (optional)
Medium Priority
Add Task
AWS session
compulsory attend
Created: 9/23/2026, 8:58:31 PM
High
Delete
Awdf pra- 1 to 5 complete
Created: 8/21/2026, 9:45:03 AM
Medium
Delete


Task "Awdf pra- 1 to 5 complete" marked as completed.
Task Manager
Practical 6 — Full Stack Integration (React + Node + Express + MongoDB)
Backend: http://localhost:5000
CORS: Enabled
Task title (required)
Description (optional)
Medium Priority
Add Task
AWS session
compulsory attend
Created: 9/23/2026, 8:58:31 PM
High
Delete
Awdf pra- 1 to 5 complete
Created: 8/21/2026, 9:45:03 AM
Medium
Delete

localhost:27017 > taskmanager > tasks

Open MongoDB shell

Documents 2 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

ADD DATA | BULK | EXPORT DATA | EXPORT CODE

25 | 1 – 2 of 2

```
_id: ObjectId('6a87d0c7a0234b2569b35af7')
title : "Awdf pra- 1 to 5 complete"
description : ""
completed : true
priority : "medium"
createdAt : ISODate('2026-08-21T04:15:03.019+00:00') August 21, 2026 at 4:15:03 AM UTC
__v : 0
```

```
_id: ObjectId('6ab3f01fe069e5d8efd48fd3')
title : "AWS session"
description : "compulsory attend"
completed : false
priority : "high"
createdAt : ISODate('2026-09-23T15:28:31.507+00:00') September 23, 2026 at 3:28:31 PM UTC
__v : 0
```

```
PS C:\Users\JIGNESH\JIGNESH USER BACKUP DATA\all Important document\Desktop\All-AWDF- Practical\task-manager-p6\
backend> npm fund
task-manager-api-p6@1.0.0
├── https://opencollective.com/express
│   └── cors@2.8.6, express@4.22.2, http-errors@2.0.1
├── https://dotenvx.com
│   └── dotenv@16.6.1
├── https://opencollective.com/mongoose
│   └── mongoose@8.24.3
└── https://opencollective.com/nodemon
    └── nodemon@3.1.14
    ├── https://paulmillr.com/funding/
    │   └── chokidar@3.6.0
    └── https://github.com/sponsors/jonschlinkert
        └── picomatch@2.3.2
    └── https://github.com/sponsors/isaacs
        └── minimatch@10.2.6
```

```
export const BASE_URL = 'http://localhost:5000';
const API_BASE = `${BASE_URL}/tasks`;

function extractErrorMessage(data, fallback) {
  if (data && Array.isArray(data.error)) return data.error.join(', ');
  if (data && data.error) return data.error;
  if (data && data.message) return data.message;
  return fallback;
}

export async function fetchTasks() {
  const res = await fetch(API_BASE);
  const data = await res.json();
  if (!res.ok) throw new Error(extractErrorMessage(data, 'Failed to fetch tasks'));
  return data.data;
}

export const getTasks = fetchTasks;
```

```
const loadTasks = useCallback(async () => {
  try {
    setLoading(true);
    setError('');
    const data = await fetchTasks();
    setTasks(data);
  } catch (err) {
    const msg = err.message || 'Failed to connect to the backend server.';
    setError(msg);
    showToast(msg, 'error');
  } finally {
    setLoading(false);
  }
}, [showToast]);

useEffect(() => {
  loadTasks();
}, [loadTasks]);
```

```
const handleCreate = async (taskData) => {
  const tempId = `temp-${Date.now()}`;
  const optimisticTask = {
    _id: tempId,
    ...taskData,
    completed: false,
    createdAt: new Date().toISOString(),
    isOptimistic: true
  };

  setTasks((prev) => [optimisticTask, ...prev]);
  setIsCreating(true);
  setError('');

  try {
    const savedTask = await createTask(taskData);

    setTasks((prev) => prev.map((t) => (t._id === tempId ? savedTask : t)));
    showToast('Task created successfully and saved in MongoDB!', 'success');
  } catch (err) {

    setTasks((prev) => prev.filter((t) => t._id !== tempId));
    const msg = err.message || 'Failed to create task';
    setError(msg);
    showToast(`Creation failed: ${msg}`, 'error');
  } finally {
    setIsCreating(false);
  }
};
```

```
const handleToggleComplete = async (task) => {
  setBusyTaskId(task._id);
  try {
    const updated = await updateTask(task._id, { completed: !task.completed });
    setTasks((prev) => prev.map((t) => (t._id === updated._id ? updated : t)));
    showToast(
      `Task "${updated.title}" marked as ${updated.completed ? 'completed' : 'pending'}.`,
      'success'
    );
    setError('');
  } catch (err) {
    const msg = err.message || 'Failed to update task';
    setError(msg);
    showToast(msg, 'error');
  } finally {
    setBusyTaskId(null);
  }
};

const handleUpdatePriority = async (task, newPriority) => {
  setBusyTaskId(task._id);
  try {
    const updated = await updateTask(task._id, { priority: newPriority });
    setTasks((prev) => prev.map((t) => (t._id === updated._id ? updated : t)));
    showToast(`Priority changed to "${newPriority}" for "${updated.title}".`, 'success');
    setError('');
  } catch (err) {
    const msg = err.message || 'Failed to update priority';
    setError(msg);
    showToast(msg, 'error');
  } finally {
    setBusyTaskId(null);
  }
};


const handleRequestDelete = (task) => {
  setDeleteModal({ isOpen: true, task, isDeleting: false });
};

const handleCancelDelete = () => {
  if (deleteModal.isDeleting) return;
  setDeleteModal({ isOpen: false, task: null, isDeleting: false });
};

const handleConfirmDelete = async () => {
  const taskToDelete = deleteModal.task;
  if (!taskToDelete) return;

  setDeleteModal((prev) => ({ ...prev, isDeleting: true }));
  try {
    await deleteTask(taskToDelete._id);
    setTasks((prev) => prev.filter((t) => t._id !== taskToDelete._id));
    showToast(`Task "${taskToDelete.title}" deleted successfully.`, 'success');
    setDeleteModal({ isOpen: false, task: null, isDeleting: false });
    setError('');
  } catch (err) {
    const msg = err.message || 'Failed to delete task';
    setError(msg);
    showToast(msg, 'error');
    setDeleteModal((prev) => ({ ...prev, isDeleting: false }));
  }
};
```

```
  return (
    <div className="app">
      {/* Supplementary Problem 3: Toast notification container */}
      <Toast toasts={toasts} onDismiss={removeToast} />

      <header className="app-header">
        <h1>Task Manager</h1>
        <p className="subtitle">Practical 6 — Full Stack Integration (React + Node + Express + MongoDB)</p>
        <div className="status-badge-row">
          <span className="api-badge">Backend: http://localhost:5000</span>
          <span className="api-badge">CORS: Enabled</span>
        </div>
      </header>

      {/* STEP 4 — Task creation form (POSTs to /tasks) */}
      <TaskForm onCreate={handleCreate} isCreating={isCreating} />

      {/* STEP 6 — Error banner with Retry button (same pattern as Practical 3) */}
      {error && (
        <div className="error-banner" role="alert">
          <strong>Error: </strong> {error}
          <button className="retry-btn" onClick={loadTasks}>Retry</button>
        </div>
      )}

      {/* STEP 6 — Loading state while GET /tasks is in flight */}
      {loading ? (
        <div className="loading-state">
          <div className="spinner"></div>
          <p className="status-text">Fetching tasks from MongoDB...</p>
        </div>
      ) : (
        <TaskList
          tasks={tasks}
          onToggleComplete={handleToggleComplete}
          onUpdatePriority={handleUpdatePriority}
          onRequestDelete={handleRequestDelete}
          busyTaskId={busyTaskId}
        />
      )}

      {/* Supplementary Problem 2 — Confirmation modal before DELETE fires */}
      <ConfirmModal
        isOpen={deleteModal.isOpen}
        title="Delete Task Confirmation"
        message={`Are you sure you want to delete "${deleteModal.task?.title}"? This will permanently remove it from MongoDB.`}
        confirmText="Yes, Delete"
        onConfirm={handleConfirmDelete}
        onCancel={handleCancelDelete}
        isDeleting={deleteModal.isDeleting}
      />
    </div>
  );
}

export default App;
```

Key Questions / Analysis	**Q1. What changes are required on the backend (CORS) to allow the React dev server to call the Express API?** **Ans:** Enable CORS in Express and allow the React server origin, such as http://localhost:5173. **Q2. Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE?** **Ans:** To keep the UI synchronized with the actual backend/database state. **Q3. What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?** **Ans:** The UI may show a successful operation even when the backend failed, causing incorrect or inconsistent data.
Learning Outcome	Students will be able to integrate a React frontend with a Node/Express/MongoDB backend into a complete, working full-stack application with proper state synchronization